

Übungsblatt 7: Tracing & Timing-Analyse

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

In dieser Übung verlassen Sie die reine Programmierung und wechseln in die Rolle des System-Analysten. Sie analysieren das fehlerhafte Multicore-System aus Termin 6 (den *Cache Coherency Failure*) mit professioneller Hardware-Trace-Technologie. Mithilfe der **TASKING BlueBox** und der Analysesoftware **winIDEA** lernen Sie, wie man ein System rückwirkungsfrei (*non-intrusive*) beobachtet, um die Grenzen des klassischen Debuggings (Beobachter-Effekt und *Heisenbugs*) zu überwinden.

Aufgabe 1: Hardware-Setup und Workspace

Im ersten Schritt bereiten wir die Hardware und die Analyse-Software vor.

1. Schalten Sie das Board aus und stecken Sie das 20-Pin-Flachbandkabel der BlueBox vorsichtig auf den Trace-Port des Boards. Schalten Sie anschließend beides ein.
2. Starten Sie die Software winIDEA und erstellen Sie einen neuen Workspace.
3. Laden Sie die `.elf`-Dateien beider Kerne (CM7 und CM4) in den Workspace, da diese die DWARF-Debug-Symbole enthalten, die Speicheradressen auf C-Code-Zeilen mappen.
4. Konfigurieren Sie die Taktfrequenz der CPU in winIDEA zwingend auf 400 MHz.

Frage: Warum ist die exakte Konfiguration der CPU-Taktfrequenz (400 MHz) in winIDEA so essenziell für die Timing-Analyse?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: OS-Awareness (FreeRTOS)

Der Hardware-Tracer zeichnet primär C-Funktionen und Speicheradressen auf. Um ein klares Gantt-Chart unserer FreeRTOS-Tasks zu erhalten, müssen wir dem Tool beibringen, das Betriebssystem zu verstehen.

1. Gehen Sie in winIDEA auf *Plugins* und aktivieren Sie *FreeRTOS*.

Frage: Erklären Sie das Konzept der OS-Awareness. Wie schafft es die BlueBox, völlig rückwirkungsfrei (ohne Code-Overhead auf der CPU) herauszufinden, welche FreeRTOS-Task gerade den Zustand *Running* besitzt?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Profiler und DWT-Tracing einrichten

Wir wollen nun die fehlerhafte Variable aus dem Shared Memory der 6. Laborübung überwachen. In Übungsblatt 6 heißt die Shared-Variable konsistent `sd` (Typ `SharedData_t`); in den Vorlesungsfolien erscheint dasselbe Struct teils noch als `ping_pong_data`. Beobachten Sie das Flag `sd.m7_ready`.

1. Öffnen Sie in winIDEA den *Analyzer / Profiler*.
2. Fügen Sie einen DWT-Data-Trace für die C-Variable `sd.m7_ready` hinzu.

Frage: Warum nutzen wir für die Überwachung dieses Flags das DWT-Modul von ARM CoreSight und loggen den Variablen-Zustand nicht einfach durch ein `printf("Flag ist jetzt %d", sd.m7_ready)`; im C-Code des Cortex-M7?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Aufzeichnung und Analyse des Cache-Fehlers

Wir führen nun die eigentliche Messung durch und beweisen den Fehler auf der Hardware.

1. Nutzen Sie das kaputte Ping-Pong-Projekt aus Termin 6, bei dem die MPU (*Memory Protection Unit*) im Code mutwillig **deaktiviert** wurde.
2. Starten Sie das Programm auf dem STM32H755. Drücken Sie in winIDEA auf *Record* (den roten Punkt) und lassen Sie die Aufnahme für wenige Sekunden laufen.
3. Beenden Sie die Aufnahme und betrachten Sie die ausgewertete Timeline im Analyzer.

Frage: Beschreiben Sie das Aha-Erlebnis im Analyzer: Was sehen Sie in den Graphen für die M4-Task, für den M7-Kern und für die DWT-Daten-Variable `sd.m7_ready`? Welches physikalische Architektur-Problem haben Sie damit live bewiesen?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)